

MEDIATOR DESIGN PATTERN

1. THE PROBLEM

When multiple objects (like users in a chat room or UI components on a screen) need to communicate, having them reference each other directly creates an unmanageable web of dependencies. If there are n objects, the system must maintain $n \times (n - 1) / 2$ connections. This tight coupling makes the system rigid and incredibly difficult to extend, as adding a single new object forces you to update the dependency lists inside every other existing object.

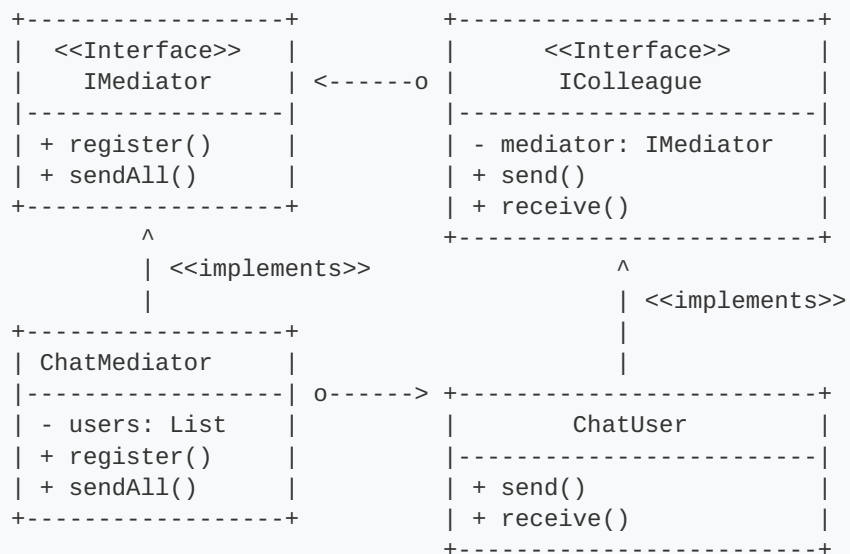
2. Intuition: Think of an airport control tower guiding airplanes. If airplanes had to talk directly to every other airplane in the sky to negotiate who lands next, it would be chaotic and dangerous. Instead, all airplanes (colleagues) only communicate with the central control tower (mediator). The tower looks at the whole picture and tells each plane when it is safe to land, keeping the planes completely independent and oblivious to one another.

3. KEY COMPONENTS

COMPONENT	ROLE	EXAMPLE
IMediator	Defines the interface for communication between colleagues.	IChatMediator
Concrete Mediator	Implements routing logic and maintains references to all colleagues.	ChatRoom
IColleague	Defines the interface for colleagues, holding a reference to the mediator.	IUser

Concrete Colleague	Sends and receives messages exclusively through its mediator reference.	ChatUser
---------------------------	---	----------

4. UML CLASS DIAGRAM



5. HOW IT WORKS — STEP BY STEP

1. Client creates a Concrete Mediator (ChatRoom).
2. Client creates multiple Concrete Colleagues (ChatUser), passing the mediator to their constructors.
3. Client registers the users with the mediator via `chatRoom.registerColleague(user)`.
4. A user wants to broadcast a message and calls `this->mediator->sendAll(message, this)`.

5. **Result:** The Mediator receives the message, encapsulates the complex routing logic, loops through its registered users list, checks constraints (like who muted whom), and calls `receive()` on the appropriate targets.

6. CODE SKELETON (C++)

```
#include <iostream>
#include <string>
#include <vector>

class IColleague; // Forward declaration

// 1. Mediator Interface
class IMediator {
public:
    virtual void registerColleague(IColleague* colleague) = 0;
    virtual void sendAll(std::string message, IColleague* sender) = 0;
    virtual ~IMediator() = default;
};

// 2. Colleague Interface
class IColleague {
protected:
    IMediator* mediator; // "has-a" relationship
    std::string name;
public:
    IColleague(IMediator* m, std::string n) : mediator(m), name(n) {}
    virtual void send(std::string message) = 0;
    virtual void receive(std::string message) = 0;
    std::string getName() { return name; }
    virtual ~IColleague() = default;
};

// 3. Concrete Mediator
class ChatMediator : public IMediator {
private:
    std::vector<IColleague*> colleagues;
public:
    void registerColleague(IColleague* colleague) override {
        colleagues.push_back(colleague);
    }

    void sendAll(std::string message, IColleague* sender) override {
```

```

        for (auto* colleague : colleagues) {
            // Do not send the message back to the original sender
            if (colleague != sender) {
                colleague->receive(sender->getName() + " says: " +
message);
            }
        }
    }
};

// 4. Concrete Colleague
class ChatUser : public IColleague {
public:
    ChatUser(IMediator* m, std::string n) : IColleague(m, n) {}

    void send(std::string message) override {
        std::cout << "\n" << name << " sending message: " << message <<
"\n";
        mediator->sendAll(message, this); // Delegate to mediator
    }

    void receive(std::string message) override {
        std::cout << name << " received -> " << message << "\n";
    }
};

// 5. Client usage
int main() {
    IMediator* chatRoom = new ChatMediator();

    IColleague* rohan = new ChatUser(chatRoom, "Rohan");
    IColleague* amit = new ChatUser(chatRoom, "Amit");
    IColleague* priya = new ChatUser(chatRoom, "Priya");

    chatRoom->registerColleague(rohan);
    chatRoom->registerColleague(amit);
    chatRoom->registerColleague(priya);

    rohan->send("Hi everyone!");
    priya->send("Hey Rohan!");

    delete priya; delete amit; delete rohan; delete chatRoom;
    return 0;
}

```

7. REAL-WORLD USE CASES

- **Chat Rooms & Multiplayer Matchmaking:** Routing chat messages or game state updates between participants without clients linking to each other directly (e.g., Chess.com servers).
- **UI Dialog Windows:** When selecting a dropdown (e.g., "Country") needs to update a list box ("States") and disable a button; a central Dialog object mediates these complex widget interactions to prevent UI components from referencing one another.
- **Air Traffic Control Systems:** Coordinating multiple independent aircraft landing and taking off through a centralized tower.

8. INTERVIEW QUICK REFERENCE

Definition: It defines an object that encapsulates how a set of objects interact, promoting loose coupling by keeping objects from referring to each other explicitly.

When to use:

- When a set of objects communicate in well-defined but complex ways, resulting in unstructured dependencies.
- When you want to reuse a component in another system but can't because it's too tightly coupled to other components.

Commonly confused with: *Observer Pattern*.

The UML diagrams look nearly identical. However, the intent is different: Observer handles one-way communication (a Subject broadcasts passive updates to observers). Mediator handles multi-directional communication (active peers relying on a middleman to route targeted messages or complex two-way conversations).

Key interview questions:

Q1: How does the Mediator pattern prevent the $n(n-1)/2$ connection problem?

A1: By centralizing communication. Instead of n objects linking to every other object (causing exponential connections), each of the n objects only maintains exactly 1 connection—to the Mediator.

Q2: What is the main drawback of the Mediator pattern?

A2: The "God Object" anti-pattern. If not careful, the Mediator class can grow incredibly large and complex as it absorbs all the routing logic and business rules of the system, becoming a massive bottleneck.

Q3: How do the Colleagues know about the Mediator?

A3: They are typically passed a reference to the Mediator interface during instantiation (via constructor injection), establishing a strict "has-a" relationship.